

From Audio to Beatmap

A signal-processing pipeline that turns any song
into a playable five-difficulty osu! beatmap set

audio file → STFT → spectral flux → autocorrelation → .osz



The problem

A **beatmap** is a timed list of things to click:

- ▶ **circles** – tap on a beat
- ▶ **sliders** – hold and follow a path
- ▶ **spinners** – spin for a sustained passage

Each needs a **time** (ms), a **position** on a 512×384 playfield, and a **type**.

Human mappers do this by ear, for hours per song.



The problem

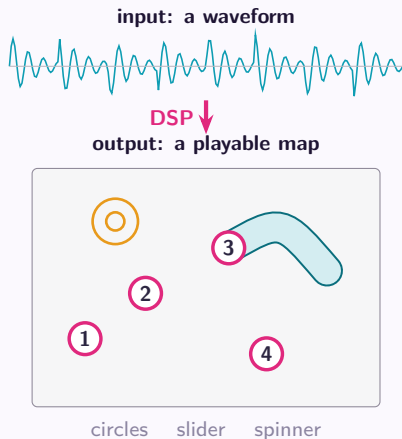
A **beatmap** is a timed list of things to click:

- ▶ **circles** – tap on a beat
- ▶ **sliders** – hold and follow a path
- ▶ **spinners** – spin for a sustained passage

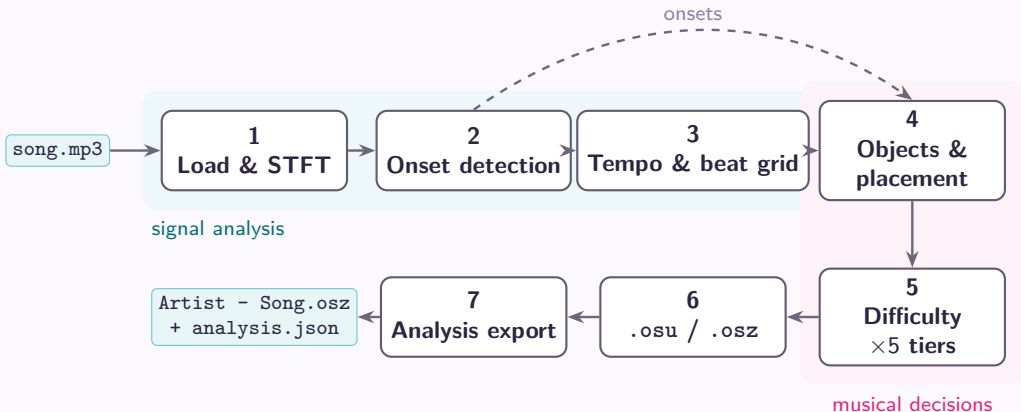
Each needs a **time** (ms), a **position** on a 512×384 playfield, and a **type**.

Human mappers do this by ear, for hours per song.

Goal: derive all three from the audio signal alone, and write a file the real game imports.

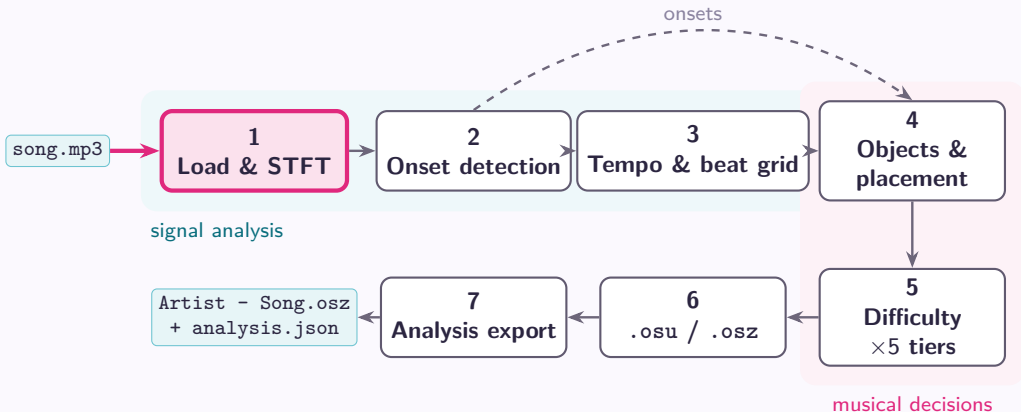


The pipeline at a glance



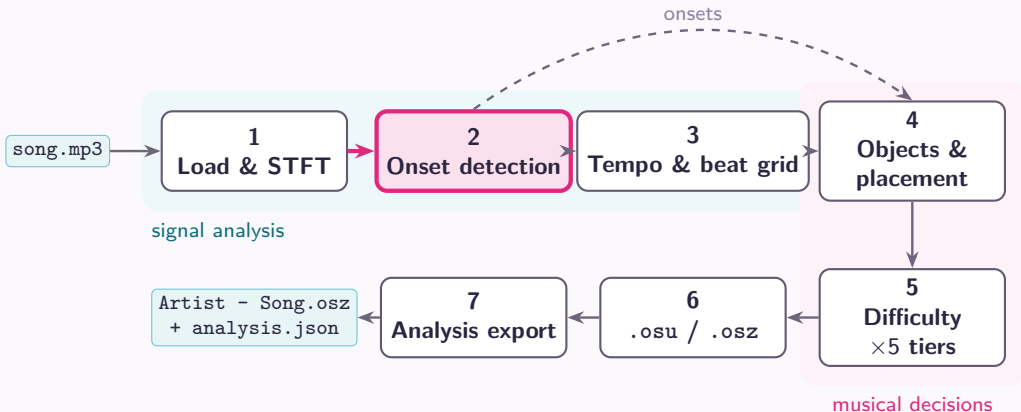
One command: `python src/main.py song.mp3` → five difficulties, Easy to Expert.

The pipeline at a glance



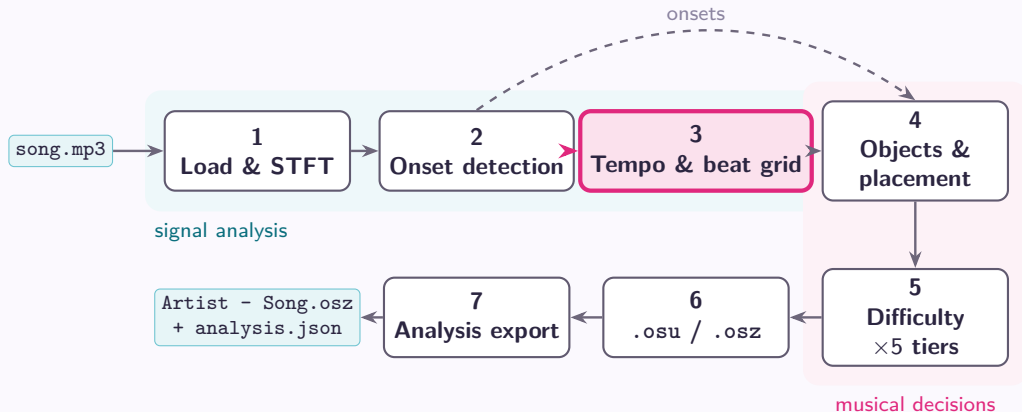
Decode, resample to 22050 Hz mono, then a short-time Fourier transform on a mel frequency scale.

The pipeline at a glance



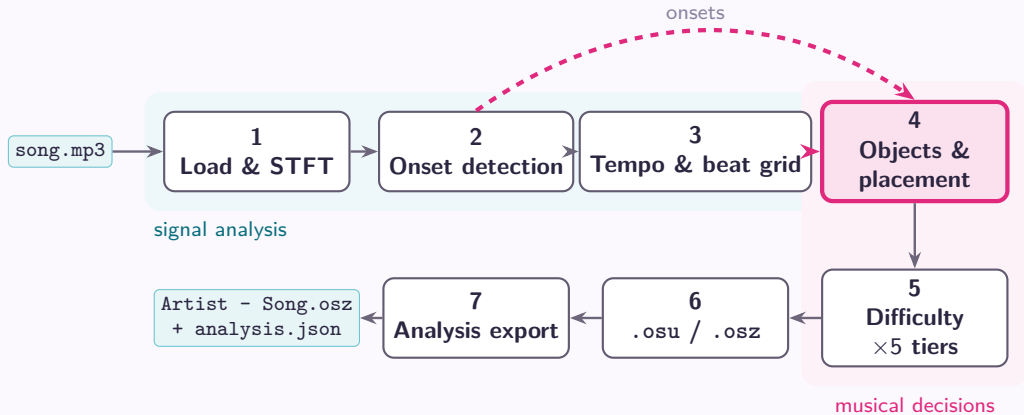
Spectral flux measures *energy appearing*; an adaptive threshold picks the note attacks.

The pipeline at a glance



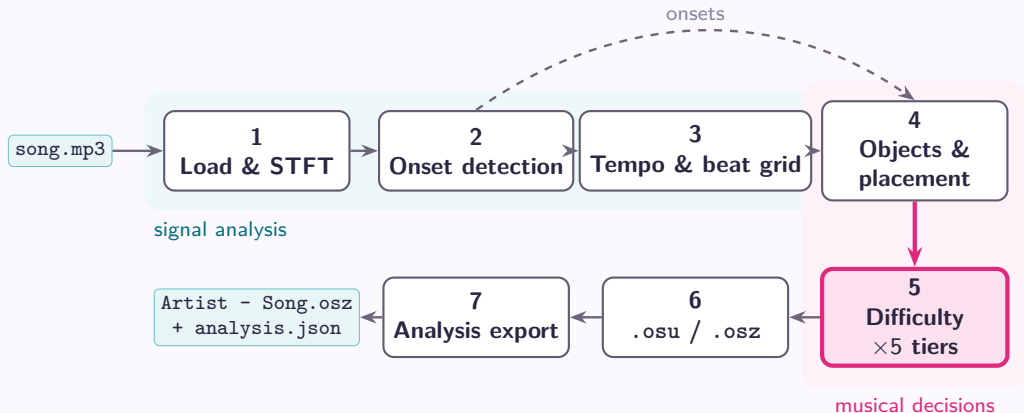
Autocorrelation of the flux finds the beat period; a pulse-train search finds its phase.

The pipeline at a glance



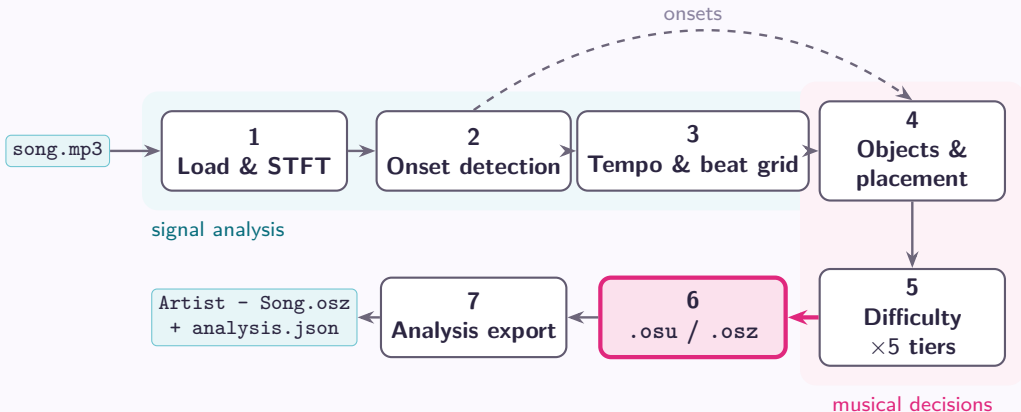
Onsets snapped to the grid become circles, sliders or spinners, placed the way a human mapper would.

The pipeline at a glance



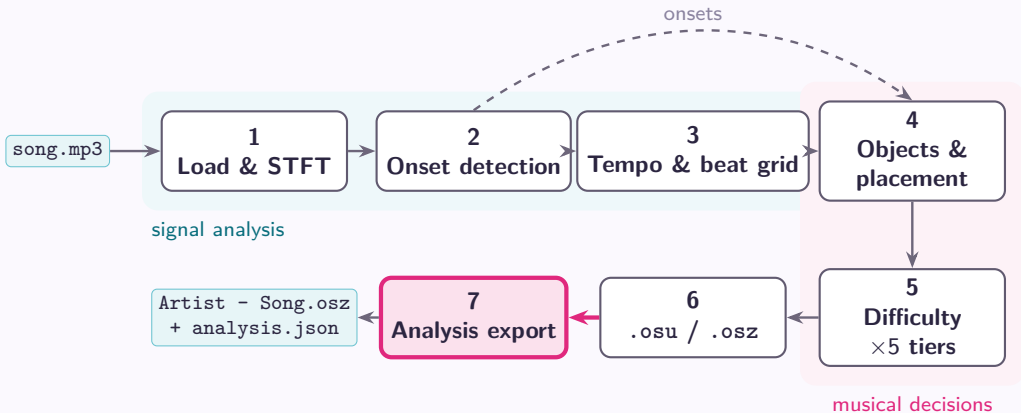
The same analysis is re-read with five parameter sets – sensitivity, grid, spacing, circle size.

The pipeline at a glance



Rendered as osu file format v14 and zipped into an .osz the game imports.

The pipeline at a glance



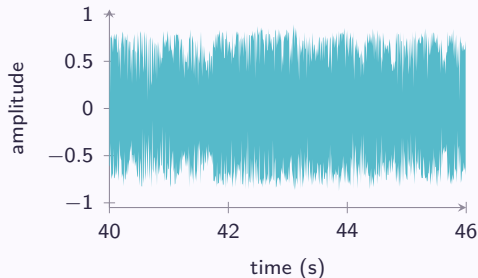
What the DSP saw (onsets, grid, spectrogram) is saved for the app's visualisation.

Running example

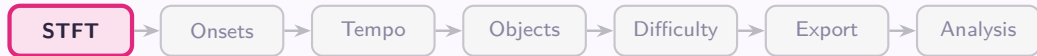
Every plot in this talk is **real output** of the generator, computed by its own functions on one song:

- ▶ **bad_apple.mp3** – 219.1 s, decoded at 22050 Hz
- ▶ zoomed plots show 40–46 s
- ▶ per-tier numbers use the **Hard** preset unless stated

The data is regenerated by `presentation/make_data.py`, so the figures cannot drift from the code.



peak-normalised waveform, 40–46 s

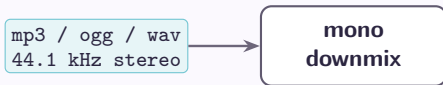


Step 1

Loading & time-frequency analysis

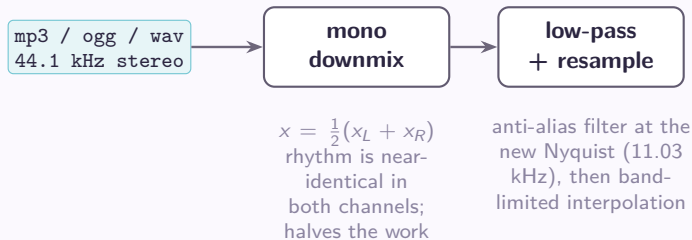
What does the song contain, and when?

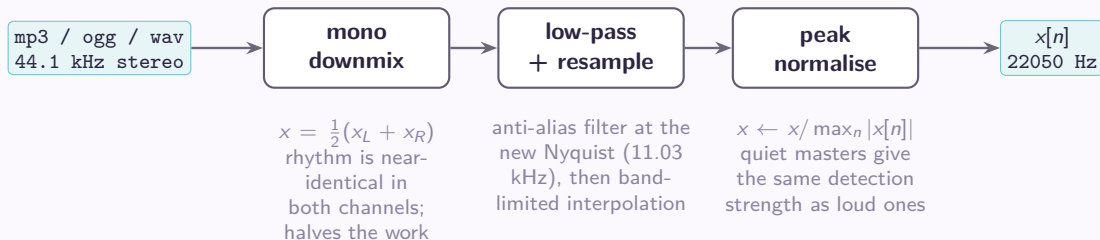
```
mp3 / ogg / wav  
44.1 kHz stereo
```

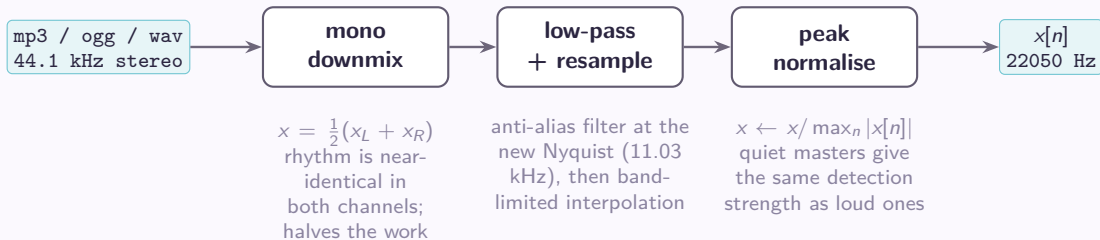


$$x = \frac{1}{2}(x_L + x_R)$$

rhythm is near-
identical in
both channels;
halves the work

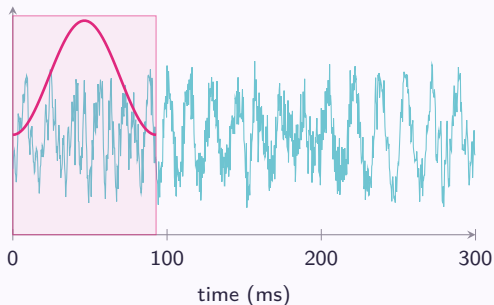






Why 22.05 kHz is enough

Onsets and beats live in how energy *changes over time*; content above 11 kHz (cymbal shimmer) adds almost nothing, and half the samples halves every later cost.

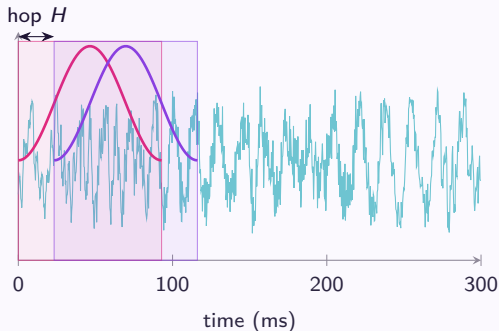


the first 300 ms, a Hann window every hop

Slide a window w along the signal;
each position gives one spectrum:

$$X[m, k] = \sum_{n=0}^{N-1} x[n+mH] w[n] e^{-j2\pi kn/N}$$

window N	2048 samples = 92.9 ms
hop H	512 samples = 23.2 ms
frame rate	43.1 spectra / s
bin width	$f_s/N = 10.8$ Hz

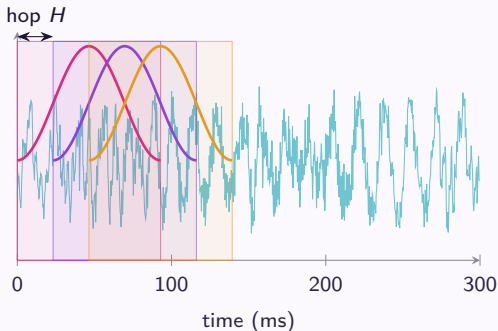


the first 300 ms, a Hann window every hop

Slide a window w along the signal;
each position gives one spectrum:

$$X[m, k] = \sum_{n=0}^{N-1} x[n+mH] w[n] e^{-j2\pi kn/N}$$

window N	2048 samples = 92.9 ms
hop H	512 samples = 23.2 ms
frame rate	43.1 spectra / s
bin width	$f_s/N = 10.8$ Hz

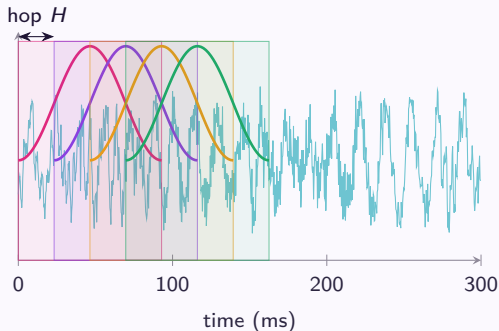


the first 300 ms, a Hann window every hop

Slide a window w along the signal;
each position gives one spectrum:

$$X[m, k] = \sum_{n=0}^{N-1} x[n+mH] w[n] e^{-j2\pi kn/N}$$

window N	2048 samples = 92.9 ms
hop H	512 samples = 23.2 ms
frame rate	43.1 spectra / s
bin width	$f_s/N = 10.8$ Hz

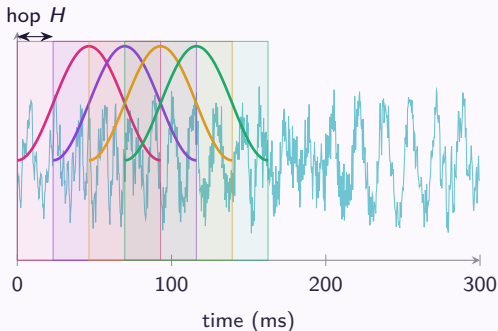


the first 300 ms, a Hann window every hop

Slide a window w along the signal;
each position gives one spectrum:

$$X[m, k] = \sum_{n=0}^{N-1} x[n+mH] w[n] e^{-j2\pi kn/N}$$

window N	2048 samples = 92.9 ms
hop H	512 samples = 23.2 ms
frame rate	43.1 spectra / s
bin width	$f_s/N = 10.8$ Hz



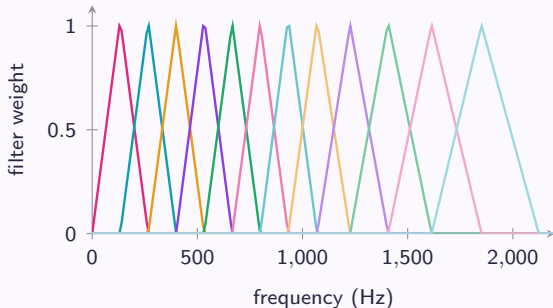
the first 300 ms, a Hann window every hop

Slide a window w along the signal;
each position gives one spectrum:

$$X[m, k] = \sum_{n=0}^{N-1} x[n+mH] w[n] e^{-j2\pi kn/N}$$

window N	2048 samples = 92.9 ms
hop H	512 samples = 23.2 ms
frame rate	43.1 spectra / s
bin width	$f_s/N = 10.8$ Hz

Trade-off: a longer window resolves pitch better but smears *when* a hit happens. 23 ms steps are finer than any hit a player can react to.

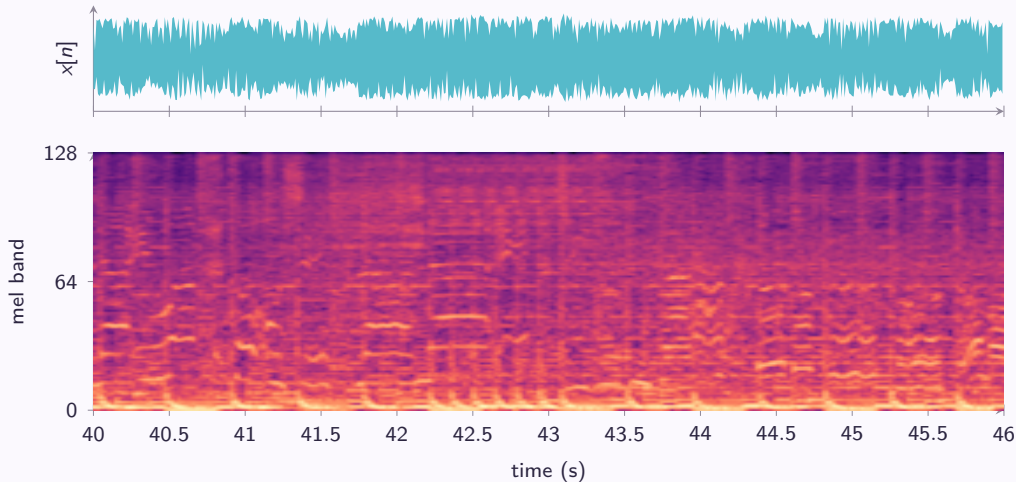


lowest filters of a mel filterbank (drawn with 24 bands)

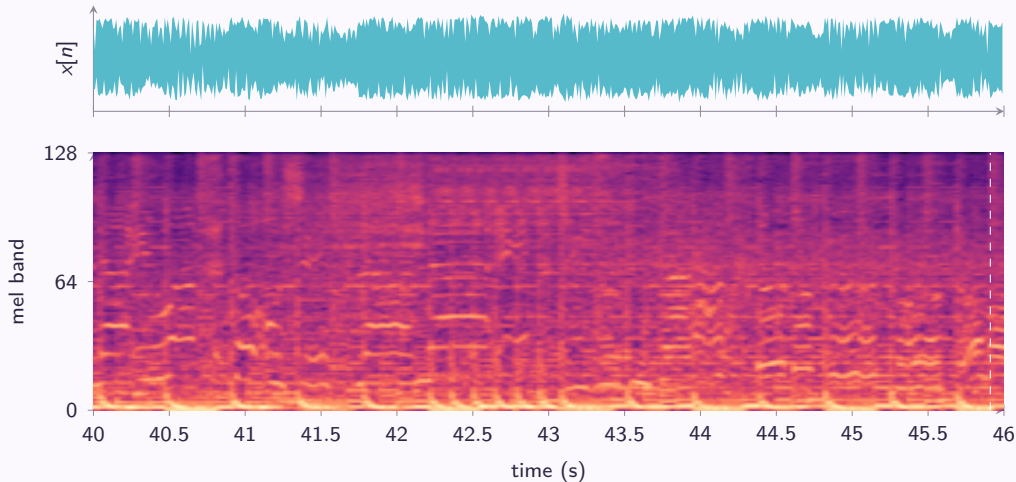
Pool the $2048/2+1$ linear bins into **128 triangular bands** spaced evenly in mel:

$$\text{mel}(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

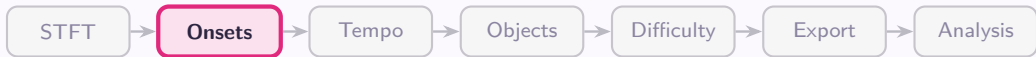
- ▶ narrow bands low (kick, bass), wide bands high
- ▶ matches pitch perception, and where rhythm lives
- ▶ power $\rightarrow$ dB for display:
 $10 \log_{10} P$



9439 frames $\times$ 128 bands for the whole song.



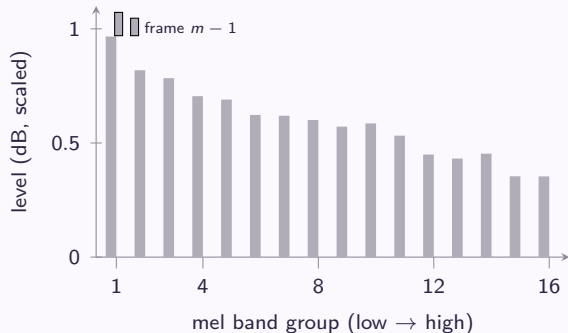
9439 frames $\times$ 128 bands for the whole song. Vertical streaks = broadband attacks: **that is what we want to detect.**



Step 2

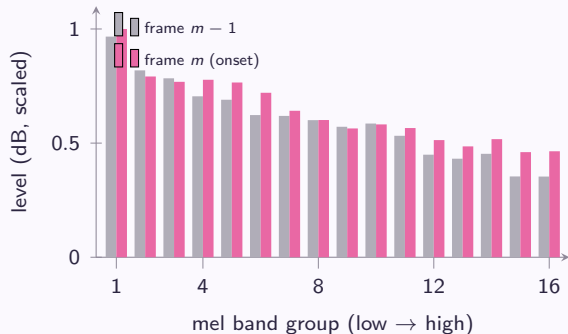
Onset detection

When does a new sound start?



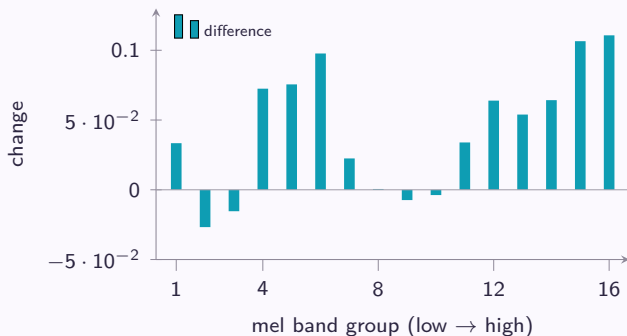
Compare each spectrum with the one before it.

the onset at 45.91 s: 11 of 16 groups get louder at once



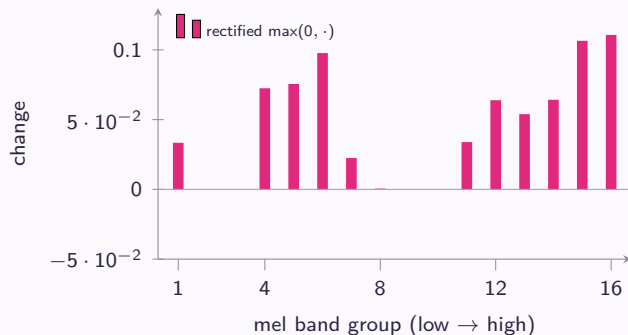
At an attack, energy jumps up across many bands at once.

the onset at 45.91 s: 11 of 16 groups get louder at once



Subtract frame by frame. Negative values are notes *decaying* – not new events.

the onset at 45.91 s: 11 of 16 groups get louder at once

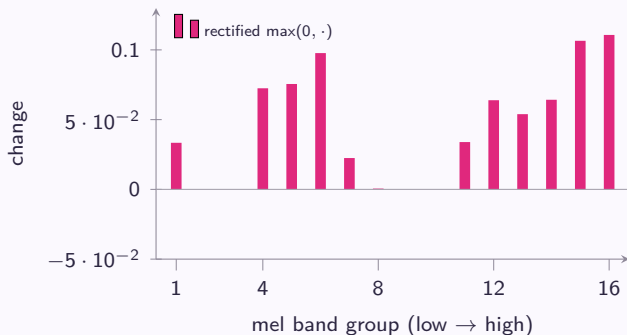


the onset at 45.91 s: 11 of 16 groups get louder at once

Keep only increases (half-wave rectify), then take the norm across bands:

$$\Delta P[m, k] = P[m, k] - P[m - 1, k]$$

$$\text{SF}[m] = \sqrt{\sum_k \max(0, \Delta P[m, k])^2}$$



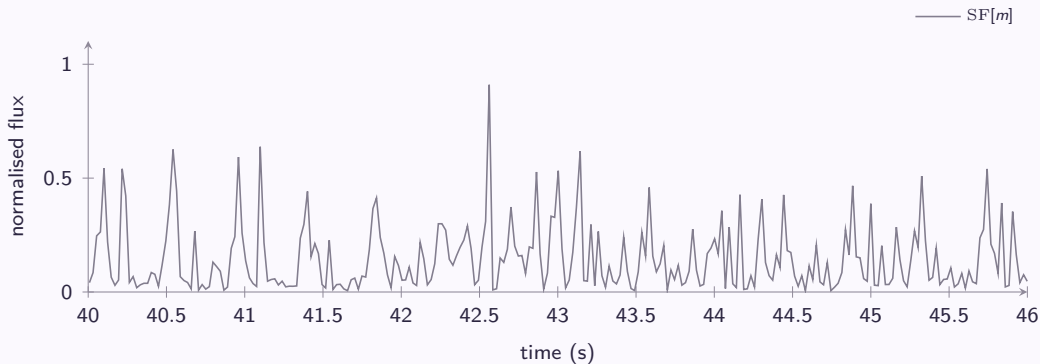
the onset at 45.91 s: 11 of 16 groups get louder at once

Keep only increases (half-wave rectify), then take the norm across bands:

$$\Delta P[m, k] = P[m, k] - P[m - 1, k]$$

$$\text{SF}[m] = \sqrt{\sum_k \max(0, \Delta P[m, k])^2}$$

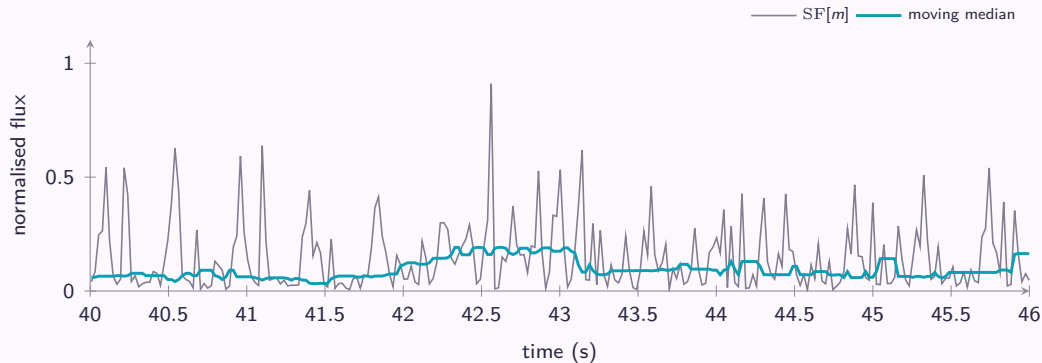
- ▶ computed on **linear power** – differences of dB values over-weight quiet noise
- ▶ one number per frame: 43.1 values / s
- ▶ then min-max normalised to $[0, 1]$



$$T[m] = \delta + \lambda \cdot \text{median}_{|i-m| \leq w} SF[i]$$

$w = 0.5 \text{ s}$, Hard: $\lambda = 1.5$, $\delta = 0.05$

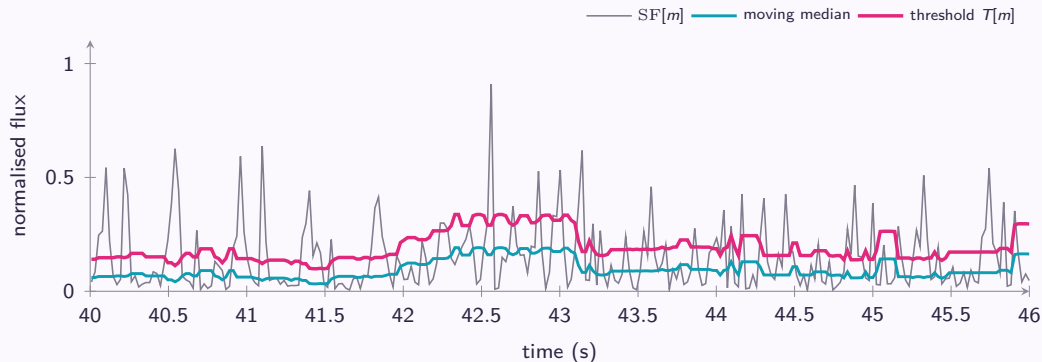
Median, not mean: the spikes themselves don't drag the floor up. It tracks the local loudness of a quiet verse or a loud drop.



$$T[m] = \delta + \lambda \cdot \text{median}_{|i-m| \leq w} SF[i]$$

$w = 0.5 \text{ s}$, Hard: $\lambda = 1.5$, $\delta = 0.05$

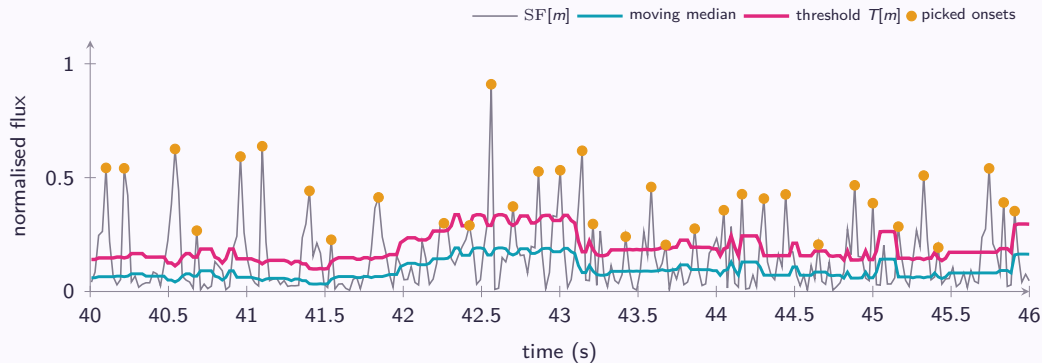
Median, not mean: the spikes themselves don't drag the floor up. It tracks the local loudness of a quiet verse or a loud drop.



$$T[m] = \delta + \lambda \cdot \text{median}_{|i-m| \leq w} SF[i]$$

$w = 0.5 \text{ s}$, Hard: $\lambda = 1.5$, $\delta = 0.05$

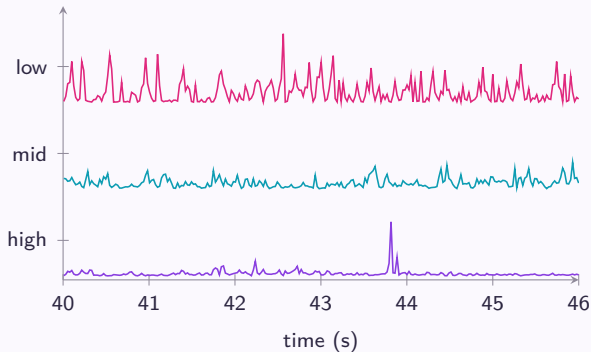
λ scales with the floor; δ is a hard floor so near-silence never turns noise into onsets (found with a synthetic noise test).



$$T[m] = \delta + \lambda \cdot \text{median}_{|i-m| \leq w} SF[i]$$

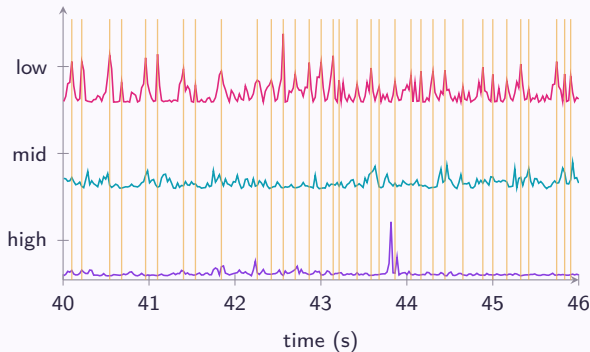
$w = 0.5 \text{ s}$, Hard: $\lambda = 1.5$, $\delta = 0.05$

Peak picking: a local maximum above $T[m]$, at least 50 ms after the last kept one (else keep the stronger). 34 onsets here, 862 in the song.



The same flux, restricted to three bands:

low	20–200 Hz	kick, bass
mid	200–2000 Hz	snare, vocals
high	2–10 kHz	hats, cymbals



The same flux, restricted to three bands:

low	20–200 Hz	kick, bass
mid	200–2000 Hz	snare, vocals
high	2–10 kHz	hats, cymbals

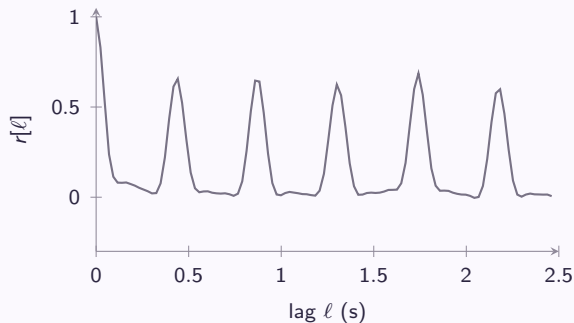
Each onset records its **dominant band** and its **strength**. They cost nothing extra and travel with the onset as provenance, all the way to the exported analysis.



Step 3

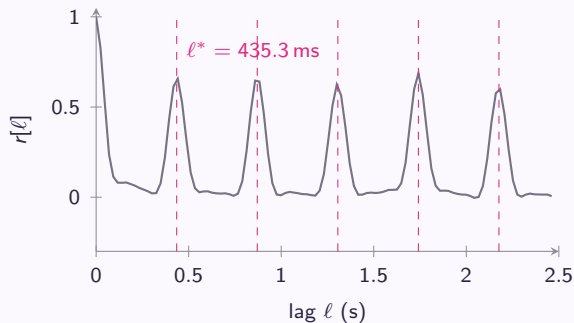
Tempo & beat grid

How fast is the music, and where is beat one?



A beat is a *repeating* ripple in the flux. Correlating the envelope with a shifted copy of itself:

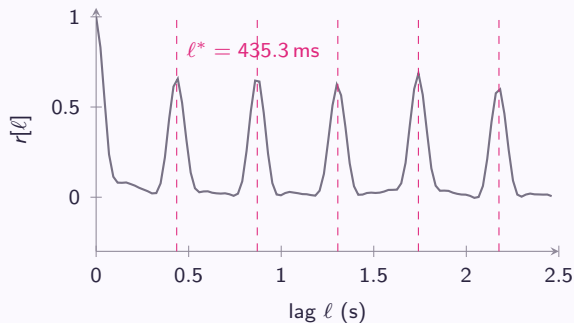
$$r[l] = \frac{\sum_m \tilde{e}[m] \tilde{e}[m+l]}{\sum_m \tilde{e}[m]^2}, \quad \tilde{e} = e - \bar{e}$$



A beat is a *repeating* ripple in the flux. Correlating the envelope with a shifted copy of itself:

$$r[l] = \frac{\sum_m \tilde{e}[m] \tilde{e}[m+l]}{\sum_m \tilde{e}[m]^2}, \quad \tilde{e} = e - \bar{e}$$

peaks at the beat period and all its multiples.



A beat is a *repeating* ripple in the flux. Correlating the envelope with a shifted copy of itself:

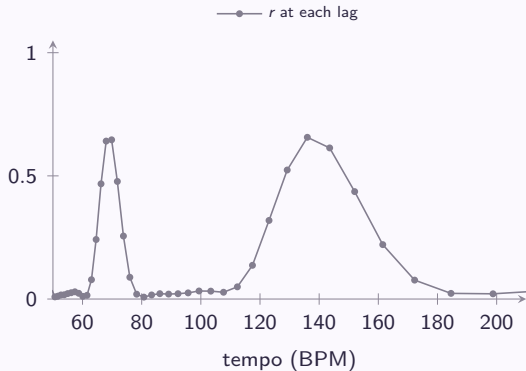
$$r[\ell] = \frac{\sum_m \tilde{e}[m] \tilde{e}[m + \ell]}{\sum_m \tilde{e}[m]^2}, \quad \tilde{e} = e - \bar{e}$$

peaks at the beat period and all its multiples. Computed with the

Wiener–Khinchin theorem, $O(n \log n)$:

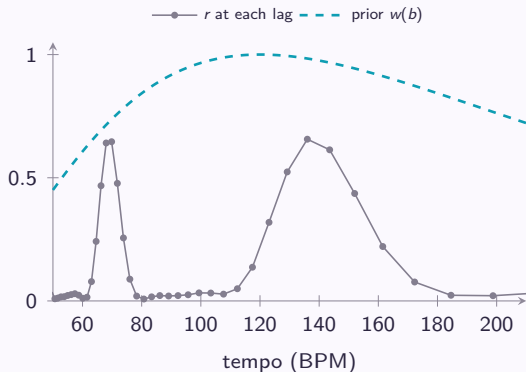
$$r = \mathcal{F}^{-1}\{|\mathcal{F}\{\tilde{e}\}|^2\}$$

(zero-padded to avoid circular wrap-around).



Search lags for **50–210 BPM** only:

$$b = \frac{60 f_s}{H\ell}$$

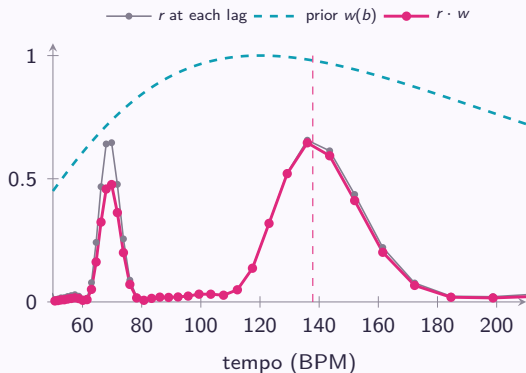


Search lags for **50–210 BPM** only:

$$b = \frac{60 f_s}{H\ell}$$

A log-normal **tempo prior** centred on 120 BPM, where human tempo perception clusters (Parncutt 1994):

$$w(b) = \exp\left(-\frac{1}{2}\left(\log_2 \frac{b}{120}\right)^2\right)$$



Search lags for **50–210 BPM** only:

$$b = \frac{60 f_s}{H \ell}$$

A log-normal **tempo prior** centred on 120 BPM, where human tempo perception clusters (Parncutt 1994):

$$w(b) = \exp\left(-\frac{1}{2}\left(\log_2 \frac{b}{120}\right)^2\right)$$

The weighted peak, refined to sub-frame precision by a parabola through its neighbours:

$$\ell^* = i + \frac{1}{2} \frac{r_{i-1} - r_{i+1}}{r_{i-1} - 2r_i + r_{i+1}}$$

⇒ **137.85 BPM** (lag 18.75 frames).

Autocorrelation peaks equally at $\frac{1}{2}\times$, $1\times$ and $2\times$ the true tempo – the classic failure of tempo estimation.

Re-score each candidate by *how well a pulse train at that tempo lands on real onset energy*:

$$\text{score}(b) = \underbrace{\frac{\text{mean envelope on the beats}}{\text{mean envelope overall}}}_{\text{phase strength}} \times w(b)$$

candidate	tempo	phase strength	prior w	score
$\frac{1}{2}\times$	68.9	1.41	0.73	1.02
$1\times$	137.8	1.38	0.98	1.35
$2\times$	275.7	1.08	0.49	0.52

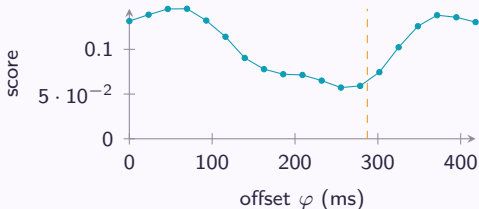
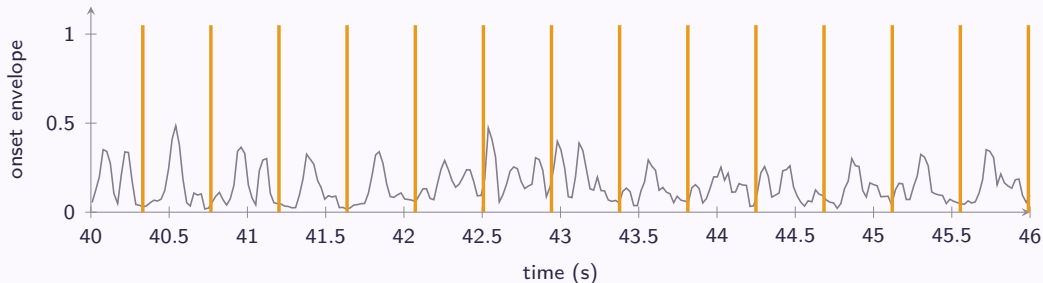
Autocorrelation peaks equally at $\frac{1}{2}\times$, $1\times$ and $2\times$ the true tempo – the classic failure of tempo estimation.

Re-score each candidate by *how well a pulse train at that tempo lands on real onset energy*:

$$\text{score}(b) = \underbrace{\frac{\text{mean envelope on the beats}}{\text{mean envelope overall}}}_{\text{phase strength}} \times w(b)$$

candidate	tempo	phase strength	prior w	score
$\frac{1}{2}\times$	68.9	1.41	0.73	1.02
$1\times$	137.8	1.38	0.98	1.35
$2\times$	275.7	1.08	0.49	0.52

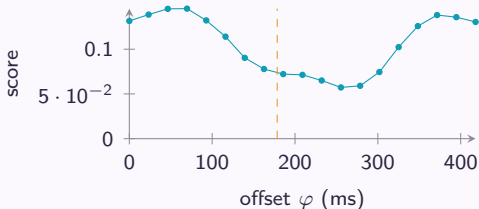
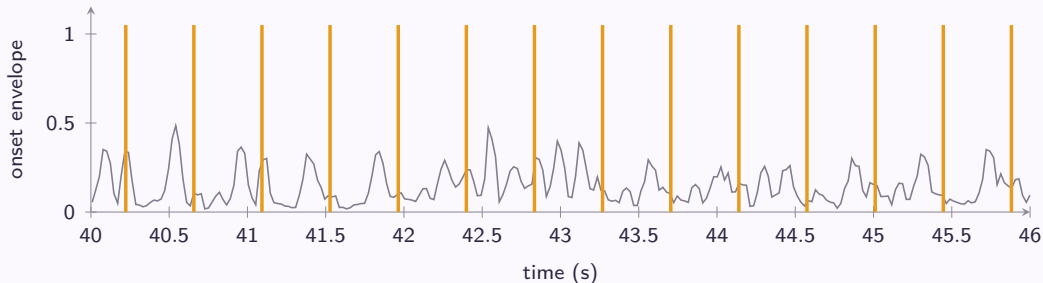
$2\times$ also falls outside the 210 BPM search range. Half-tempo aligns almost as well (every other beat still lands on a hit) – the prior breaks the tie. **Result: 137.85 BPM.**



Slide a pulse train of period P across one beat; score each offset by the envelope it collects (linear interpolation keeps a fractional P from drifting):

$$\varphi^* = \arg \max_{\varphi \in [0, P)} \frac{1}{N} \sum_n e(\varphi + nP)$$

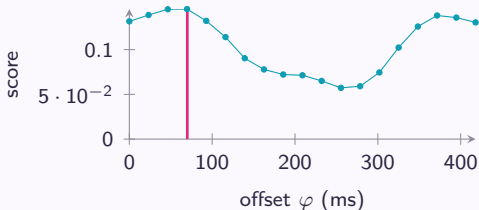
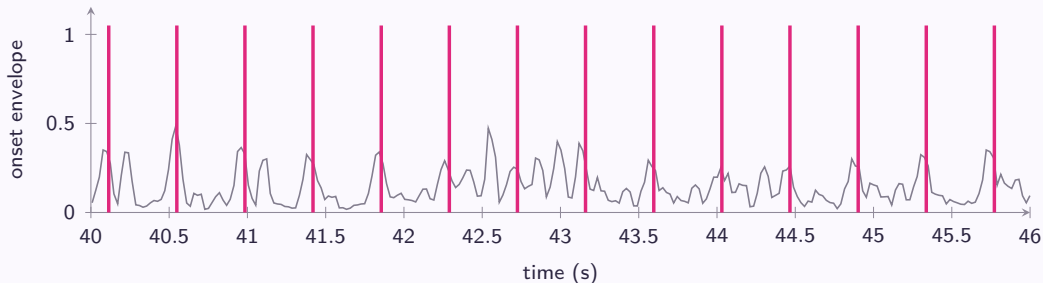
Half a beat off: pulses fall between hits.



Slide a pulse train of period P across one beat; score each offset by the envelope it collects (linear interpolation keeps a fractional P from drifting):

$$\varphi^* = \arg \max_{\varphi \in [0, P)} \frac{1}{N} \sum_n e(\varphi + nP)$$

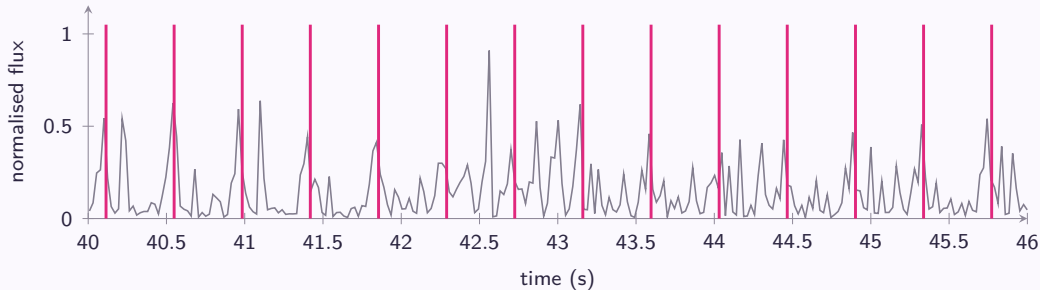
A quarter off: better, still missing.



Slide a pulse train of period P across one beat; score each offset by the envelope it collects (linear interpolation keeps a fractional P from drifting):

$$\varphi^* = \arg \max_{\varphi \in [0, P)} \frac{1}{N} \sum_n e(\varphi + nP)$$

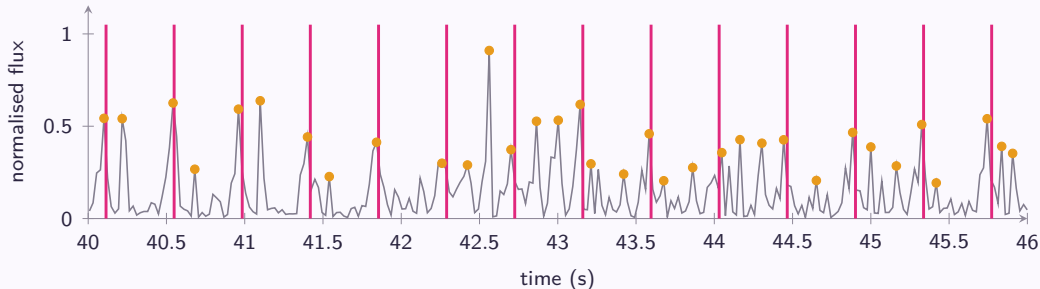
$\varphi^* = 70 \text{ ms}$: every pulse lands on a hit (phase strength 1.38).



$$t_n = \varphi^* + n \cdot \frac{60}{b}, \quad n = 0, 1, \dots$$

504 beats, one every 435.3 ms, autocorrelation confidence 0.66.

One global tempo, deliberately: rhythm-game music is machine-timed, and a constant grid is what makes a map feel “snapped”.



$$t_n = \varphi^* + n \cdot \frac{60}{b}, \quad n = 0, 1, \dots$$

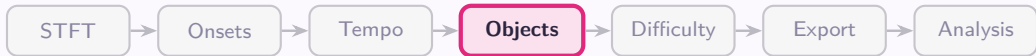
504 beats, one every 435.3 ms, autocorrelation confidence 0.66.

One global tempo, deliberately: rhythm-game music is machine-timed, and a constant grid is what makes a map feel “snapped”.

In the file this is one line:

```
70,435.26,4,1,0,70,1,0
```

offset ms, beat length ms, ..., uninherited



Step 4

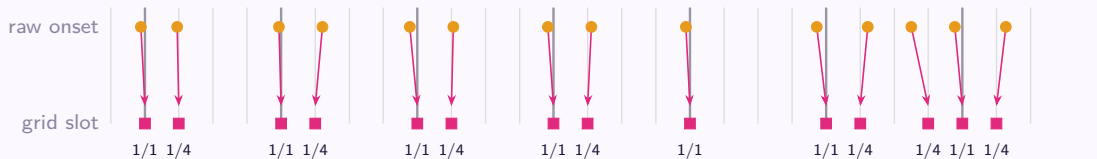
Hit objects & placement

Which object, and where on screen?



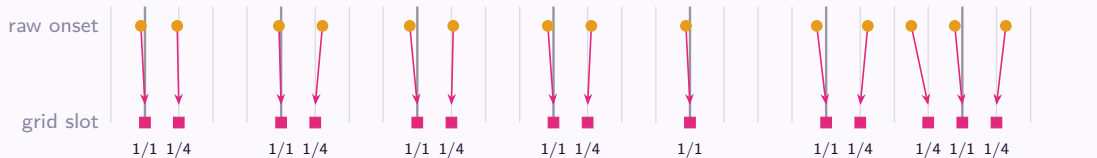
$$k = \text{round}\left(\frac{t - \varphi^*}{P/d}\right), \quad \hat{t} = \varphi^* + k \frac{P}{d}$$

$d = 4$ here (1/4-beat grid); the label is the rhythmic snap.



$$k = \text{round}\left(\frac{t - \varphi^*}{P/d}\right), \quad \hat{t} = \varphi^* + k \frac{P}{d}$$

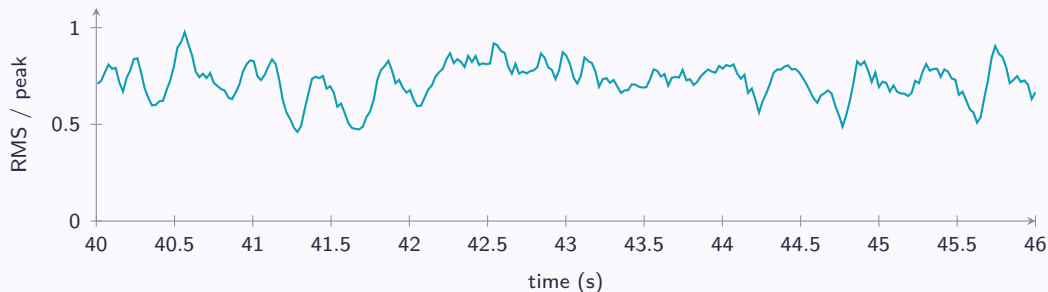
$d = 4$ here (1/4-beat grid); the label is the rhythmic snap.



$$k = \text{round}\left(\frac{t - \varphi^*}{P/d}\right), \quad \hat{t} = \varphi^* + k \frac{P}{d}$$

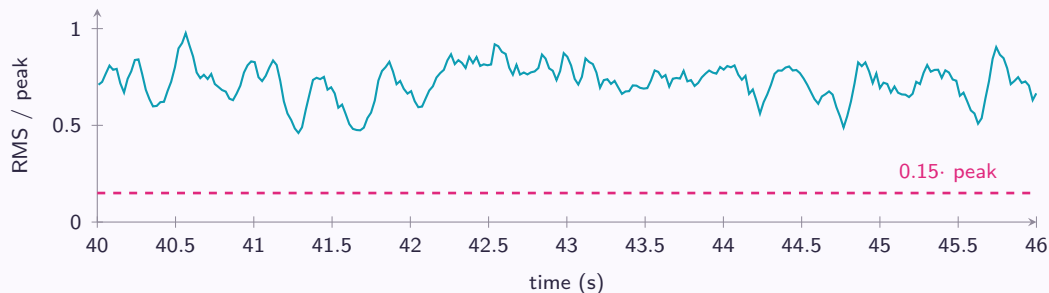
$d = 4$ here (1/4-beat grid); the label is the rhythmic snap.

- ▶ removes detector jitter ($\pm$ one hop)
- ▶ two onsets in one slot $\rightarrow$ keep the stronger
(77 merged over the song)
- ▶ each tier then **thins** greedily to its minimum spacing



Short-time RMS energy, per frame:

$$\text{RMS}[m] = \sqrt{\frac{1}{N} \sum_n x[n + mH]^2}$$



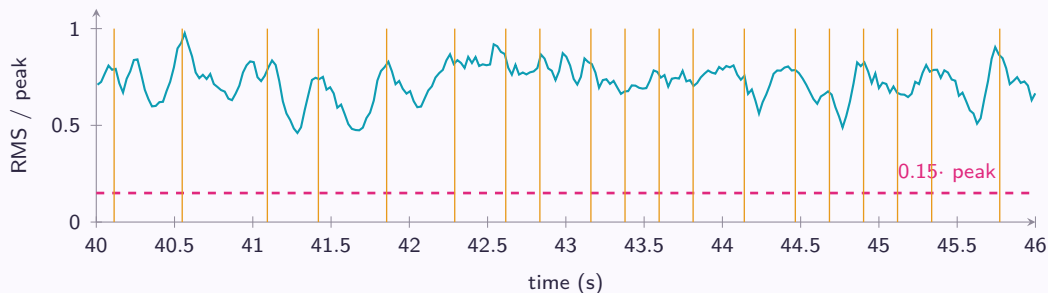
Short-time RMS energy, per frame:

$$\text{RMS}[m] = \sqrt{\frac{1}{N} \sum_n x[n + mH]^2}$$

Sustain ratio of the gap after an onset:

$$s = \frac{1}{|G|} \sum_{m \in G} \mathbf{1}[\text{RMS}[m] > 0.15 \max \text{RMS}]$$

G : the frames between this onset and the next.



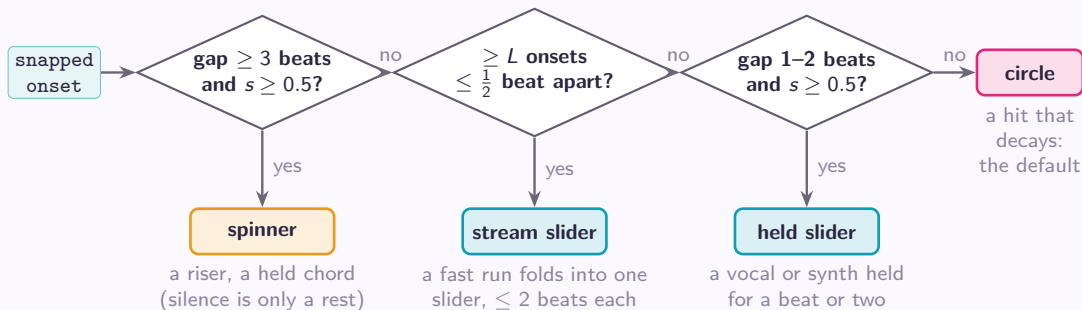
Short-time RMS energy, per frame:

$$\text{RMS}[m] = \sqrt{\frac{1}{N} \sum_n x[n + mH]^2}$$

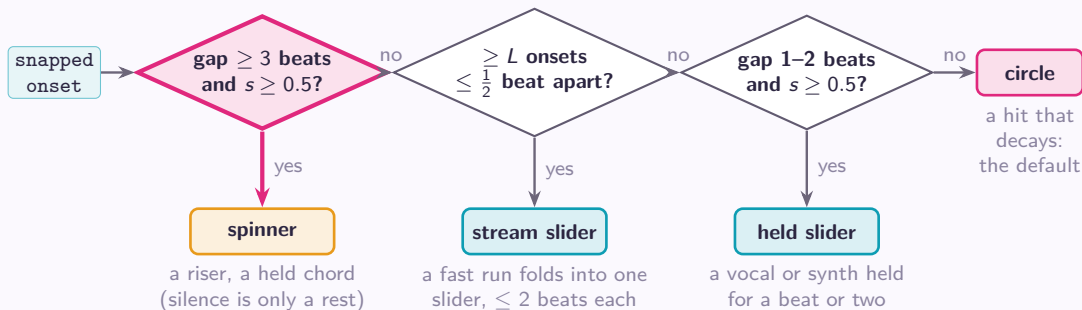
Sustain ratio of the gap after an onset:

$$s = \frac{1}{|G|} \sum_{m \in G} \mathbf{1}[\text{RMS}[m] > 0.15 \max \text{RMS}]$$

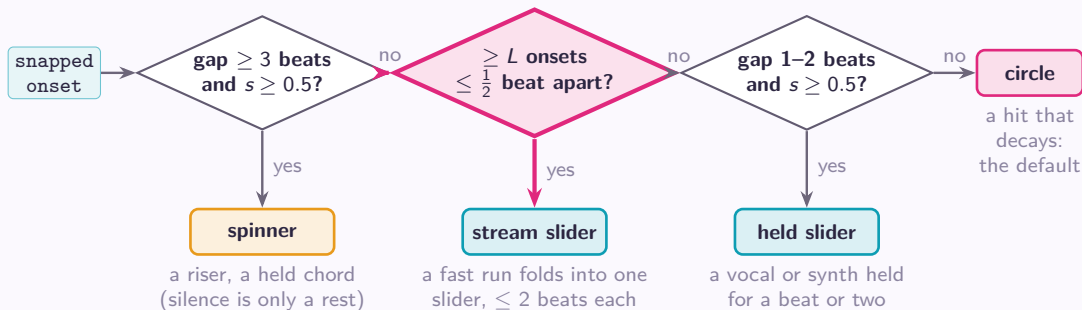
G : the frames between this onset and the next. $s \approx 1$: the note rings on (hold). $s \approx 0$: it decayed (tap).



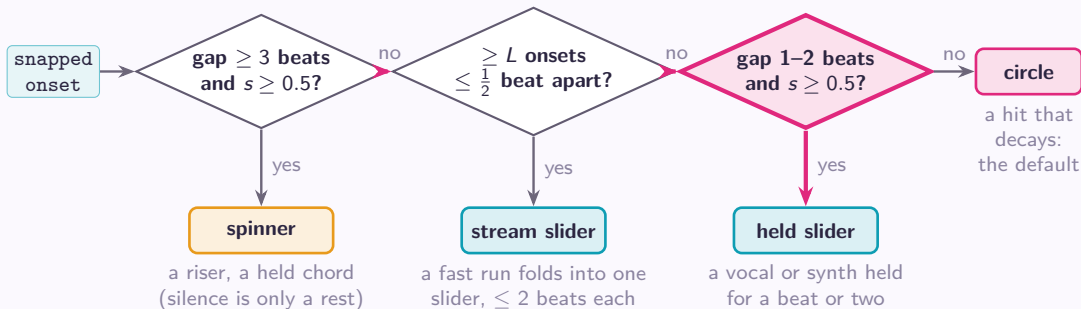
“Clean” gaps only: if a thinned-out onset sat inside the gap, the sound was re-struck, so it cannot be a hold. L (stream length) rises with the tier: Easy folds every run, Expert keeps each note as a circle.



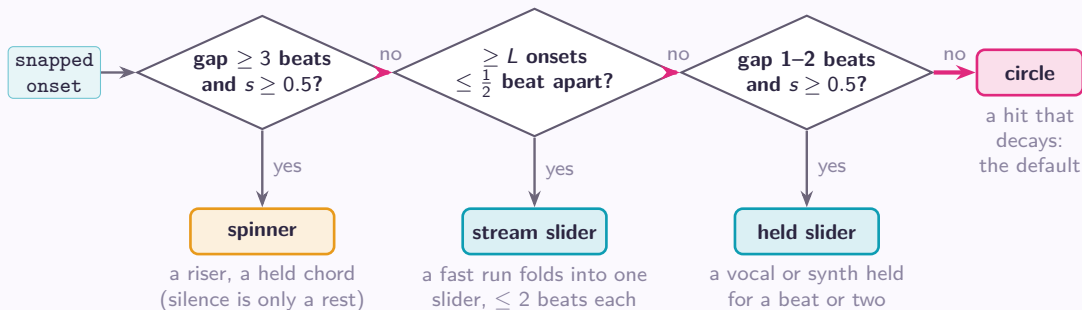
“Clean” gaps only: if a thinned-out onset sat inside the gap, the sound was re-struck, so it cannot be a hold. L (stream length) rises with the tier: Easy folds every run, Expert keeps each note as a circle.



“Clean” gaps only: if a thinned-out onset sat inside the gap, the sound was re-struck, so it cannot be a hold. L (stream length) rises with the tier: Easy folds every run, Expert keeps each note as a circle.



“Clean” gaps only: if a thinned-out onset sat inside the gap, the sound was re-struck, so it cannot be a hold. L (stream length) rises with the tier: Easy folds every run, Expert keeps each note as a circle.



“Clean” gaps only: if a thinned-out onset sat inside the gap, the sound was re-struck, so it cannot be a hold. L (stream length) rises with the tier: Easy folds every run, Expert keeps each note as a circle.

Mappers place the next object a fixed number of **circle diameters** per beat of elapsed time – the editor's *distance snap*. We do the same:

$$d = D \cdot \sigma \cdot \Delta b, \quad D = 2(54.4 - 4.48 \text{ CS})$$

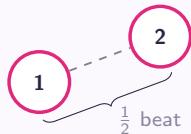
- ▶ σ : diameters per beat (0.85 Easy $\rightarrow$ 2.10 Expert)
- ▶ spacing is **linear in time**: a 1/1 gap reads as twice a 1/2 gap
- ▶ spacing tracks circle size, so patterns read the same at any CS

1

Mappers place the next object a fixed number of **circle diameters** per beat of elapsed time – the editor's *distance snap*. We do the same:

$$d = D \cdot \sigma \cdot \Delta b, \quad D = 2(54.4 - 4.48 \text{ CS})$$

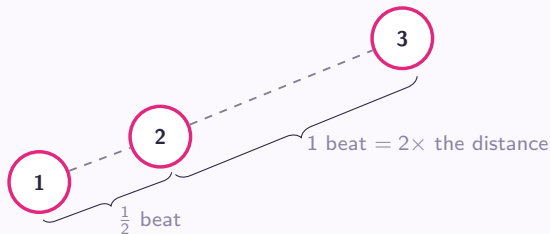
- ▶ σ : diameters per beat (0.85 Easy $\rightarrow$ 2.10 Expert)
- ▶ spacing is **linear in time**: a 1/1 gap reads as twice a 1/2 gap
- ▶ spacing tracks circle size, so patterns read the same at any CS



Mappers place the next object a fixed number of **circle diameters** per beat of elapsed time – the editor's *distance snap*. We do the same:

$$d = D \cdot \sigma \cdot \Delta b, \quad D = 2 (54.4 - 4.48 \text{ CS})$$

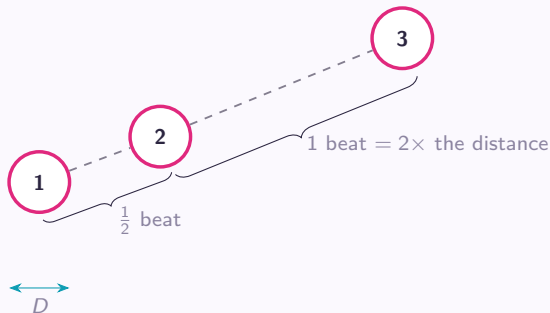
- ▶ σ : diameters per beat (0.85 Easy $\rightarrow$ 2.10 Expert)
- ▶ spacing is **linear in time**: a 1/1 gap reads as twice a 1/2 gap
- ▶ spacing tracks circle size, so patterns read the same at any CS

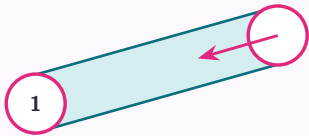


Mappers place the next object a fixed number of **circle diameters** per beat of elapsed time – the editor's *distance snap*. We do the same:

$$d = D \cdot \sigma \cdot \Delta b, \quad D = 2 (54.4 - 4.48 \text{ CS})$$

- ▶ σ : diameters per beat (0.85 Easy $\rightarrow$ 2.10 Expert)
- ▶ spacing is **linear in time**: a 1/1 gap reads as twice a 1/2 gap
- ▶ spacing tracks circle size, so patterns read the same at any CS

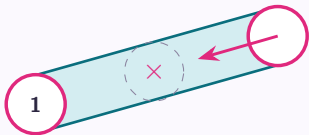




magenta: the heading the flow asks for – a sharp turn back

A virtual cursor walks the objects in time order:

1. **Flow:** the heading turns only a little across short gaps (streams arc smoothly), freely across long ones (jumps). Max turn 40° Easy $\rightarrow 130^\circ$ Expert.
2. **Never bury:** $\geq 0.75 D$ from the previous object ($0.65 D$ in a stream).
3. **Stay off slider bodies:** clear the *drawn curve*, not just its ends.

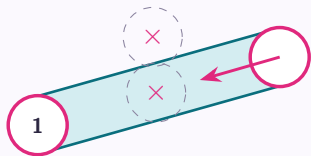


candidate hits the slider body: rotate and retry

A virtual cursor walks the objects in time order:

1. **Flow:** the heading turns only a little across short gaps (streams arc smoothly), freely across long ones (jumps). Max turn 40° Easy $\rightarrow 130^\circ$ Expert.
2. **Never bury:** $\geq 0.75 D$ from the previous object ($0.65 D$ in a stream).
3. **Stay off slider bodies:** clear the *drawn curve*, not just its ends.

Candidates are tried at $0^\circ, \pm 25^\circ, \pm 50^\circ, \dots$ off the heading; near a wall the heading is nudged back to the centre.

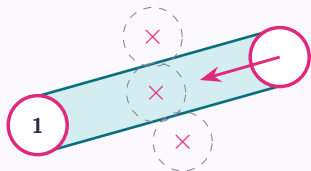


candidate hits the slider body: rotate and retry

A virtual cursor walks the objects in time order:

1. **Flow:** the heading turns only a little across short gaps (streams arc smoothly), freely across long ones (jumps). Max turn 40° Easy $\rightarrow 130^\circ$ Expert.
2. **Never bury:** $\geq 0.75 D$ from the previous object ($0.65 D$ in a stream).
3. **Stay off slider bodies:** clear the *drawn curve*, not just its ends.

Candidates are tried at $0^\circ, \pm 25^\circ, \pm 50^\circ, \dots$ off the heading; near a wall the heading is nudged back to the centre.

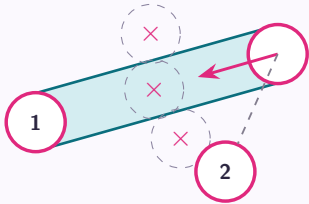


candidate hits the slider body: rotate and retry

A virtual cursor walks the objects in time order:

1. **Flow:** the heading turns only a little across short gaps (streams arc smoothly), freely across long ones (jumps). Max turn 40° Easy $\rightarrow 130^\circ$ Expert.
2. **Never bury:** $\geq 0.75 D$ from the previous object ($0.65 D$ in a stream).
3. **Stay off slider bodies:** clear the *drawn curve*, not just its ends.

Candidates are tried at $0^\circ, \pm 25^\circ, \pm 50^\circ, \dots$ off the heading; near a wall the heading is nudged back to the centre.



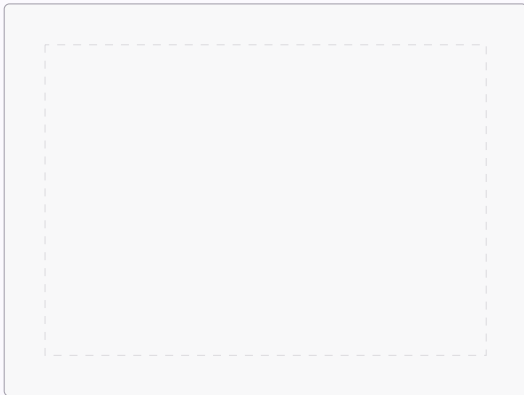
first candidate that passes every rule

A virtual cursor walks the objects in time order:

1. **Flow:** the heading turns only a little across short gaps (streams arc smoothly), freely across long ones (jumps). Max turn 40° Easy $\rightarrow 130^\circ$ Expert.
2. **Never bury:** $\geq 0.75 D$ from the previous object ($0.65 D$ in a stream).
3. **Stay off slider bodies:** clear the *drawn curve*, not just its ends.

Candidates are tried at $0^\circ, \pm 25^\circ, \pm 50^\circ, \dots$ off the heading; near a wall the heading is nudged back to the centre.

512 × 384 osu!px

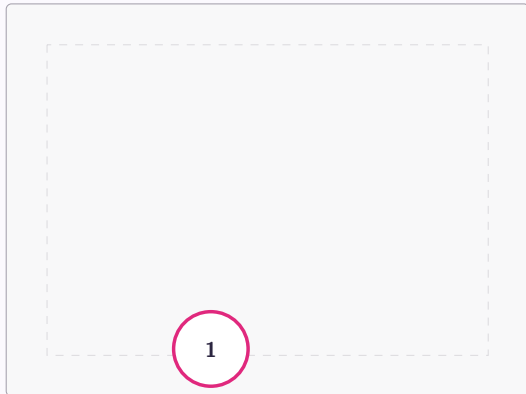


Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

512 × 384 osu!px



Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

512 × 384 osu!px

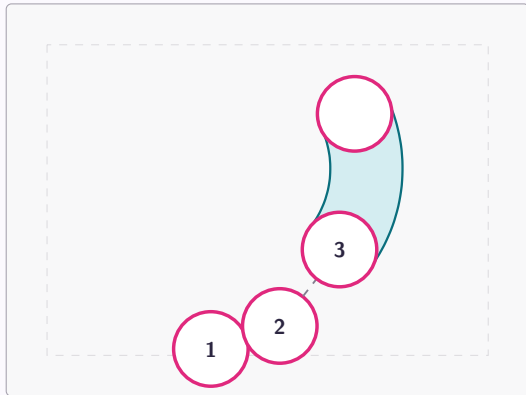


Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

512 × 384 osu!px

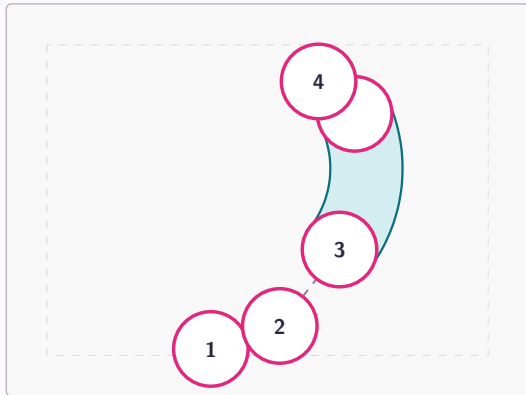


Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

512 × 384 osu!px

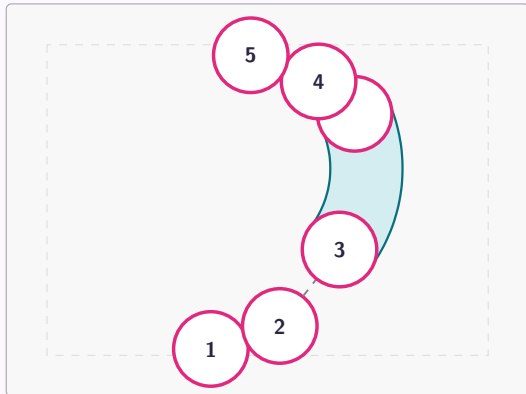


Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

512 × 384 osu!px

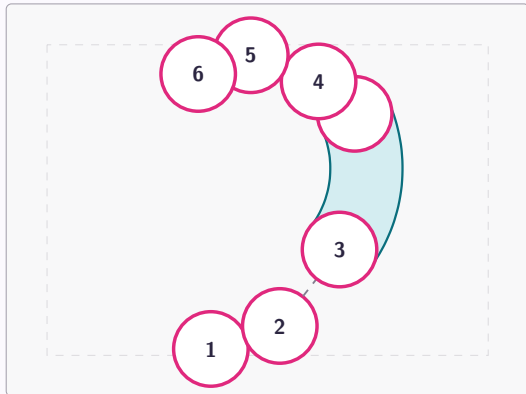


Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

512 × 384 osu!px

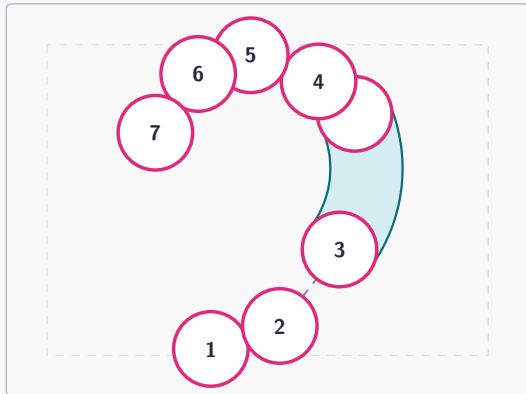


Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

512 × 384 osu!px



Hard, 7 consecutive objects of `bad_apple.mp3` in play order, drawn at true scale (circle radius 36.5 px).

- ▶ dashed line: the cursor path the player's hand takes
- ▶ the slider's next object leaves from its *end*, off to one side of the body
- ▶ a stream keeps curving the same way

Only objects within one approach time (~ 1 s) are on screen together.

straight (L, all tiers)



Timing is fixed first, shape second.
The game derives a slider's duration from its length:

$$\text{beats} = \frac{L \cdot \text{slides}}{100 \cdot \text{SM} \cdot \text{SV}}$$

so L is solved for the beats Step 4 wanted.

straight (L, all tiers)



arc (P, all tiers)



Timing is fixed first, shape second.
The game derives a slider's duration from its length:

$$\text{beats} = \frac{L \cdot \text{slides}}{100 \cdot \text{SM} \cdot \text{SV}}$$

so L is solved for the beats Step 4 wanted.

straight (L, all tiers)



arc (P, all tiers)



S-curve (B, Normal+)



Timing is fixed first, shape second.
The game derives a slider's duration from its length:

$$\text{beats} = \frac{L \cdot \text{slides}}{100 \cdot \text{SM} \cdot \text{SV}}$$

so L is solved for the beats Step 4 wanted.

straight (L, all tiers)



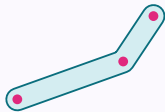
arc (P, all tiers)



S-curve (B, Normal+)



corner (L, Hard+)



Timing is fixed first, shape second.
The game derives a slider's duration from its length:

$$\text{beats} = \frac{L \cdot \text{slides}}{100 \cdot \text{SM} \cdot \text{SV}}$$

so L is solved for the beats Step 4 wanted.

straight (L, all tiers)



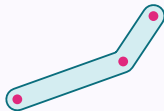
arc (P, all tiers)



S-curve (B, Normal+)



corner (L, Hard+)



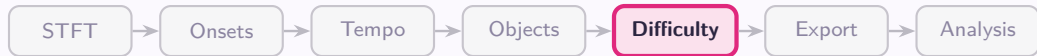
Timing is fixed first, shape second.
The game derives a slider's duration from its length:

$$\text{beats} = \frac{L \cdot \text{slides}}{100 \cdot \text{SM} \cdot \text{SV}}$$

so L is solved for the beats Step 4 wanted.

Every curve must pass: no kink at the head, stays on the playfield, never closer to recent objects than its own head, leaves room to exit. Otherwise the straight slider is used.

Measured on six songs: buried objects
 $69 \rightarrow 33$.



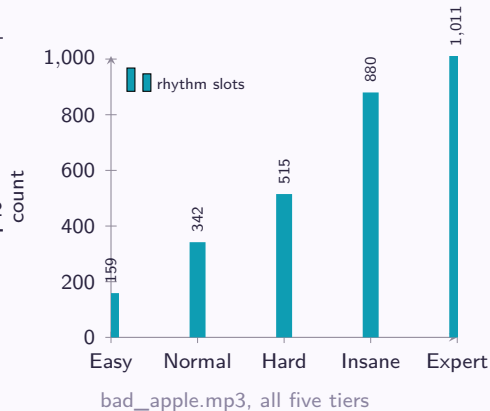
Step 5

Difficulty scaling

One analysis, five maps

knob	Easy	Hard	Expert
λ / δ	2.2 / .08	1.5 / .05	1.1 / .03
snap grid	1/1	1/4	1/4
min spacing (beats)	2.0	0.5	0.25
stream length L	4	6	999
distance σ	0.85	1.3	2.10
max turn	40°	75°	130°
AR / OD / CS	4 / 3 / 3.2	7.5 / 6 / 4	9.3 / 8.5 / 4.2

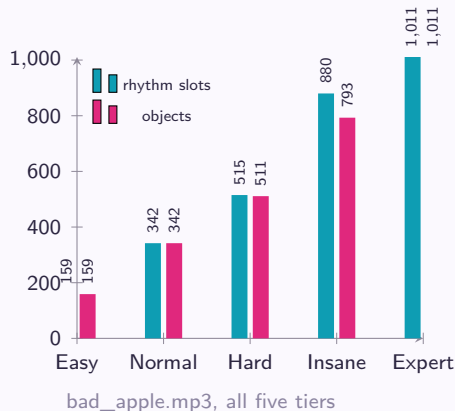
full table: `configs/difficulty_presets.yaml`

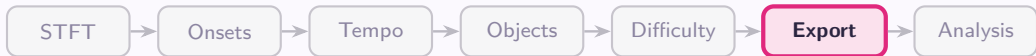


knob	Easy	Hard	Expert
λ / δ	2.2 / .08	1.5 / .05	1.1 / .03
snap grid	1/1	1/4	1/4
min spacing (beats)	2.0	0.5	0.25
stream length L	4	6	999
distance σ	0.85	1.3	2.10
max turn	40°	75°	130°
AR / OD / CS	4 / 3 / 3.2	7.5 / 6 / 4	9.3 / 8.5 / 4.2

full table: `configs/difficulty_presets.yaml`

The **DSP** knobs (λ , δ) decide how many onsets exist; the **musical** knobs decide how many survive and how they move.





Step 6

Export

A file the real game accepts

```
[Difficulty]
HPDrainRate:5
CircleSize:4
OverallDifficulty:6
ApproachRate:7.5
SliderMultiplier:1.4
SliderTickRate:1

[TimingPoints]
70,435.260746,4,1,0,70,1,0

[HitObjects]
69,192,40114,1,0,0:0:0:0:
326,240,41855,2,0,P|352:175|340:105,1,140.00,0|0,0:0|0:0,0:0:0:0:
256,192,89189,8,0,90930,0:0:0:0:
```

► [Difficulty]: the tier's preset

```
[Difficulty]
HPDrainRate:5
CircleSize:4
OverallDifficulty:6
ApproachRate:7.5
SliderMultiplier:1.4
SliderTickRate:1

[TimingPoints]
70,435.260746,4,1,0,70,1,0

[HitObjects]
69,192,40114,1,0,0:0:0:0:
326,240,41855,2,0,P|352:175|340:105,1,140.00,0|0,0:0|0:0,0:0:0:
256,192,89189,8,0,90930,0:0:0:0:
```

- ▶ [Difficulty]: the tier's preset
- ▶ [TimingPoints]: *one* red line – offset 70 ms, beat 435.3 ms (Step 3)

```
[Difficulty]
HPDrainRate:5
CircleSize:4
OverallDifficulty:6
ApproachRate:7.5
SliderMultiplier:1.4
SliderTickRate:1

[TimingPoints]
70,435.260746,4,1,0,70,1,0

[HitObjects]
69,192,40114,1,0,0:0:0:0:
326,240,41855,2,0,P|352:175|340:105,1,140.00,0|0,0:0|0:0,0:0:0:
256,192,89189,8,0,90930,0:0:0:0:
```

- ▶ [Difficulty]: the tier's preset
- ▶ [TimingPoints]: *one* red line – offset 70 ms, beat 435.3 ms (Step 3)
- ▶ [HitObjects]: one record per object
x,y,time,type,...; type bits 1 circle, 2 slider, 8 spinner, +4 new combo

```
[Difficulty]
HPDrainRate:5
CircleSize:4
OverallDifficulty:6
ApproachRate:7.5
SliderMultiplier:1.4
SliderTickRate:1

[TimingPoints]
70,435.260746,4,1,0,70,1,0

[HitObjects]
69,192,40114,1,0,0:0:0:0:
326,240,41855,2,0,P|352:175|340:105,1,140.00,0|0,0:0|0:0,0:0:0:0:
256,192,89189,8,0,90930,0:0:0:0:
```

- ▶ [Difficulty]: the tier's preset
- ▶ [TimingPoints]: *one* red line – offset 70 ms, beat 435.3 ms (Step 3)
- ▶ [HitObjects]: one record per object
x,y,time,type,...; type bits 1 circle, 2 slider, 8 spinner, +4 new combo
- ▶ a slider: curve letter + anchors, slides, length $\Rightarrow \frac{140 \cdot 1}{100 \cdot 1.4} = 1$ beat

```
[Difficulty]
HPDrainRate:5
CircleSize:4
OverallDifficulty:6
ApproachRate:7.5
SliderMultiplier:1.4
SliderTickRate:1

[TimingPoints]
70,435.260746,4,1,0,70,1,0

[HitObjects]
69,192,40114,1,0,0:0:0:0:
326,240,41855,2,0,P|352:175|340:105,1,140.00,0|0,0:0:0:0:0:0:
256,192,89189,8,0,90930,0:0:0:0:
```

- ▶ [Difficulty]: the tier's preset
- ▶ [TimingPoints]: *one* red line – offset 70 ms, beat 435.3 ms (Step 3)
- ▶ [HitObjects]: one record per object
x,y,time,type,...; type bits 1 circle, 2 slider, 8 spinner, +4 new combo
- ▶ a slider: curve letter + anchors, slides, length $\Rightarrow \frac{140 \cdot 1}{100 \cdot 1.4} = 1$ beat

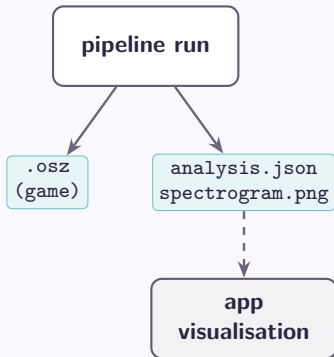
Audio + five .osu files, zipped →
Artist - Song.osz.

osu!(lazer) refuses malformed files, so a clean import *is* the end-to-end test.

The .osu format stores *where* and *when*, never *why*. An extra export keeps the evidence:

- ▶ every onset with its **strength** and **dominant band**
- ▶ the beat grid, BPM and confidence
- ▶ each object's **snap** (1/1, 1/2, 1/4) and source onset
- ▶ the mel spectrogram as an image (max-pooled, so single-frame attacks survive)

This is what the app visualises – shown in the demo.



1 Synthetic signals

Click tracks at a known BPM; held tones; 1/4 bursts. Exact expectations: every click found, tempo recovered, the held tone becomes a spinner, the burst folds into a slider.

2 Listening tests

Sonification: mix a click at every onset, or a metronome at every beat, and listen. Drift and double triggers are obvious by ear – the standard check in music information retrieval.

3 Play-testing

Import into osu!(lazer) and play. Real feedback (overlaps, circles hidden under slider bodies) became automated placement tests.

pytest: 256 backend tests across every stage

STFT + mel

92.9 ms windows, 23.2 ms hop

STFT + mel

92.9 ms windows, 23.2 ms hop

Spectral flux

half-wave rectified, linear power

STFT + mel

92.9 ms windows, 23.2 ms hop

Spectral flux

half-wave rectified, linear power

Adaptive threshold

$\delta + \lambda \cdot$ moving median

STFT + mel

92.9 ms windows, 23.2 ms hop

Spectral flux

half-wave rectified, linear power

Adaptive threshold

$\delta + \lambda \cdot$ moving median

Autocorrelation

Wiener–Khinchin + tempo prior

STFT + mel	92.9 ms windows, 23.2 ms hop
Spectral flux	half-wave rectified, linear power
Adaptive threshold	$\delta + \lambda \cdot$ moving median
Autocorrelation	Wiener–Khinchin + tempo prior
Octave + phase	pulse-train alignment

STFT + mel	92.9 ms windows, 23.2 ms hop
Spectral flux	half-wave rectified, linear power
Adaptive threshold	$\delta + \lambda \cdot$ moving median
Autocorrelation	Wiener–Khinchin + tempo prior
Octave + phase	pulse-train alignment
Sustain (RMS)	circle / slider / spinner

STFT + mel	92.9 ms windows, 23.2 ms hop
Spectral flux	half-wave rectified, linear power
Adaptive threshold	$\delta + \lambda \cdot$ moving median
Autocorrelation	Wiener–Khinchin + tempo prior
Octave + phase	pulse-train alignment
Sustain (RMS)	circle / slider / spinner
Distance snap	mapper rules as geometry

STFT + mel	92.9 ms windows, 23.2 ms hop
Spectral flux	half-wave rectified, linear power
Adaptive threshold	$\delta + \lambda \cdot$ moving median
Autocorrelation	Wiener–Khinchin + tempo prior
Octave + phase	pulse-train alignment
Sustain (RMS)	circle / slider / spinner
Distance snap	mapper rules as geometry

137.85 BPM, 862 onsets
→ 5 difficulties,
159 to 1011 objects

Limits & next steps: variable tempo, a real star-rating model, hitsounds, per-section slider velocity.

Demo

the app, end to end